Table des matières

Cours 07 — Les codes 1
Étape 2 — Le fait incrémental, et le filtre de tout le monde · MARTS 1
Étape 3 — Ce que le 13 juin a perdu · Fiabiliser et faire tourner 4
Étape 4 — L'instant de l'export, de la RAW au fait · Fiabiliser et faire tourner 5

Cours 07 — Les codes

Cours 7/9 · La Brûlerie — Formation Analytics Engineering

STAGING	INTERMEDIATE	MARTS
reprendre la `RAW`, une vue par table	joindre, clarifier, préparer l'étoile	concevoir et livrer les étoiles
✓ ici · Fiabiliser et faire tourner, ce qui traverse les trois couches : les tests, l'incrémental, l'intégration continue	✓	✓

À ouvrir après avoir essayé. Le code garde les commentaires numérotés du cours, pour comparer ligne par ligne. Chaque fichier est rangé sous son chemin, et chaque requête du notebook sous son étape ; les requêtes se tapent dans le notebook de DuckDB (`duckdb -ui`, lancé depuis `Projet_Brulerie`), une par cellule, sous celles qui attachent et détachent l'entrepôt. Un fichier qui existait déjà se montre en entier, dans son nouvel état ; le fait des ventes se montre deux fois, avec le filtre de l'étape 2, puis avec celui de l'étape 4.

Étape 2 — Le fait incrémental, et le filtre de tout le monde · MARTS

02_ae/models/marts/_marts.yml

```yaml
# 02_ae/models/marts/_marts.yml : les dimensions et les trois faits, décrits et testés
models:
  # 1. les dimensions : une clé unique et toujours remplie
  - name: dim_date
    description: "Les jours du 1er mai 2023 au 30 juin 2025, un par ligne, fabriqués sans source."
    columns:
      - name: date_jour
        description: "La clé : la date elle-même."
        data_tests:
          - unique
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
      - not_null
- name: dim_produit
  description: "Un produit du catalogue par ligne, au 2025-06-30."
  columns:
    - name: cle_produit
      description: "La clé de substitution, md5 du numéro du produit."
      data_tests:
        - unique
        - not_null
    # 2. une liste fermée, et son not_null à côté
    - name: categorie
      data_tests:
        - not_null
        - accepted_values:
            arguments:
              values: ['ACCESSOIRE', 'BOISSON_CHAUDE', 'BOISSON_FROIDE', 'GRAINS', 'SANDWICH', 'VIENNOISERIE']
- name: dim_membre
  description: "Un membre du programme de fidélité par ligne, sans rien de personnel."
  columns:
    - name: cle_membre
      description: "La clé de substitution, md5 du numéro de la carte."
      data_tests:
        - unique
        - not_null
# 3. le fait, bâti par passes : chaque clé recalculée existe dans sa dimension
- name: fct_ventes
  description: "Le fait des ventes, une ligne par ligne d'un ticket payé."
  config:
    materialized: incremental
    incremental_strategy: delete+insert
    unique_key: [ticket_id, ligne]
  columns:
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_tests:
        - not_null
```


© 2026 iAlexMG. Tous droits réservés.

```yaml
        - relationships:
            arguments:
              to: ref('dim_produit')
              field: cle_produit
    - name: cle_membre
      description: "Vide pour un ticket sans carte de membre."
      data_tests:
        - relationships:
            arguments:
              to: ref('dim_membre')
              field: cle_membre
# 4. le fait d'accumulation : une ligne par lot, et chaque clé dans sa dimension
- name: fct_lots
  description: "Le fait d'accumulation des lots du torréfacteur, une ligne par lot, un jalon par colonne."
  columns:
    - name: lot_id
      description: "Le numéro du lot : le grain du fait."
      data_tests:
        - unique
        - not_null
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_date')
              field: date_jour
    - name: cle_produit
      data_tests:
        - not_null
        - relationships:
            arguments:
              to: ref('dim_produit')
              field: cle_produit
# 5. le fait sans mesure : chaque clé dans sa dimension, et le statut dans une liste fermée
- name: fct_consentements
  description: "Le fait sans mesure des consentements, une ligne par décision d'un membre."
  columns:
    - name: cle_date
      data_tests:
        - not_null
        - relationships:
```


© 2026 iAlexMG. Tous droits réservés.

```
              arguments:
                to: ref('dim_date')
                field: date_jour
        - name: cle_membre
          data_tests:
            - not_null
            - relationships:
                arguments:
                  to: ref('dim_membre')
                  field: cle_membre
        - name: statut
          description: "Ce que dit la décision : accordé, refusé ou retiré."
          data_tests:
            - not_null
            - accepted_values:
                arguments:
                  values: ['ACCORDE', 'REFUSE', 'RETIRE']
```

02_ae/models/marts/fct_ventes.sql, avec le filtre sur le jour de vente

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait
    ticket_id,
    ligne,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
-- 4. une passe ne garde que ce qui est arrivé depuis la précédente : les jours de vente plus récents
{% if is_incremental() %}
where date_vente > (select max(cle_date) from {{ this }})
{% endif %}
```

Étape 3 — Ce que le 13 juin a perdu · Fiabiliser et faire tourner

© 2026 iAlexMG. Tous droits réservés.

3.3 Les lignes que le fait n'a pas

```
-- 1. les lignes vendues que le fait n'a pas : dans la vue, absentes du fait
select v.date_vente, count(distinct v.ticket_id) as tickets, count(*) as lignes,
       sum(v.montant) as montant
from brulerie.int_ventes as v
left join brulerie.fct_ventes as f on f.ticket_id = v.ticket_id and f.ligne = v.ligne
where f.ticket_id is null
group by v.date_vente;
```

Étape 4 — L'instant de l'export, de la `RAW` au fait · Fiabiliser et faire tourner

02_ae/models/staging/stg_lignes.sql

```
-- 02_ae/models/staging/stg_lignes.sql : les lignes des tickets, typées, une par ligne de ticket
select
    -- 1. typer : le texte de la RAW devient des nombres, sous le même nom
    ticket_id::integer as ticket_id,
    ligne::integer as ligne,
    produit_id::integer as produit_id,
    description,
    quantite::integer as quantite,
    prix_unitaire::decimal(10, 2) as prix_unitaire,
    rabais::decimal(10, 2) as rabais,
    code_rabais,
    code_taxe,
    montant::decimal(10, 2) as montant,
    -- 2. l'instant de l'export, à la fin du nom du fichier : AAAAMMJJHHMM
    strptime(replace(split_part(filename, '_', 3), '.csv', ''), '%Y%m%d%H%M') as exporte_le
-- 3. lire la RAW par sa source
from {{ source('brulerie', 'raw_lignes') }}
-- 4. tirer l'état courant : de chaque ligne de ticket, garder l'export le plus récent
qualify row_number() over (partition by ticket_id, ligne order by filename desc) = 1
```

02_ae/models/intermediate/int_ventes.sql

```
-- 02_ae/models/intermediate/int_ventes.sql : les lignes des tickets payés, avec leur date et leur membre
select
    -- 1. la ligne, avec la date et le membre de son ticket, et l'instant de son export
    l.ticket_id,
    l.ligne,
```


© 2026 iAlexMG. Tous droits réservés.

```
    t.horodatage::date as date_vente,
    t.membre_id,
    l.produit_id,
    l.quantite,
    l.prix_unitaire,
    l.rabais,
    l.montant,
    l.exporte_le
-- 2. les lignes et leur ticket, lus par ref()
from {{ ref('stg_lignes') }} as l
inner join {{ ref('stg_tickets') }} as t on t.ticket_id = l.ticket_id
-- 3. ce qui est une vente : un ticket payé ; les annulés sortent, le remboursement reste
where t.statut = 'PAYE'
```

02_ae/models/marts/fct_ventes.sql

```
-- 02_ae/models/marts/fct_ventes.sql : le fait des ventes, une ligne par ligne d'un ticket payé
select
    -- 1. une clé vers chaque dimension, calculée comme dans la dimension
    date_vente as cle_date,
    md5(produit_id::varchar) as cle_produit,
    md5(membre_id) as cle_membre,
    -- 2. le ticket et sa ligne, gardés dans le fait, et l'instant de l'export qui l'a livrée
    ticket_id,
    ligne,
    exporte_le,
    -- 3. les mesures
    quantite,
    quantite * prix_unitaire as montant_brut,
    rabais,
    montant
from {{ ref('int_ventes') }}
-- 4. une passe ne garde que les lignes d'un export plus récent que le dernier déjà rangé
{% if is_incremental() %}
where exporte_le > (select max(exporte_le) from {{ this }})
{% endif %}
```


© 2026 iAlexMG. Tous droits réservés.